

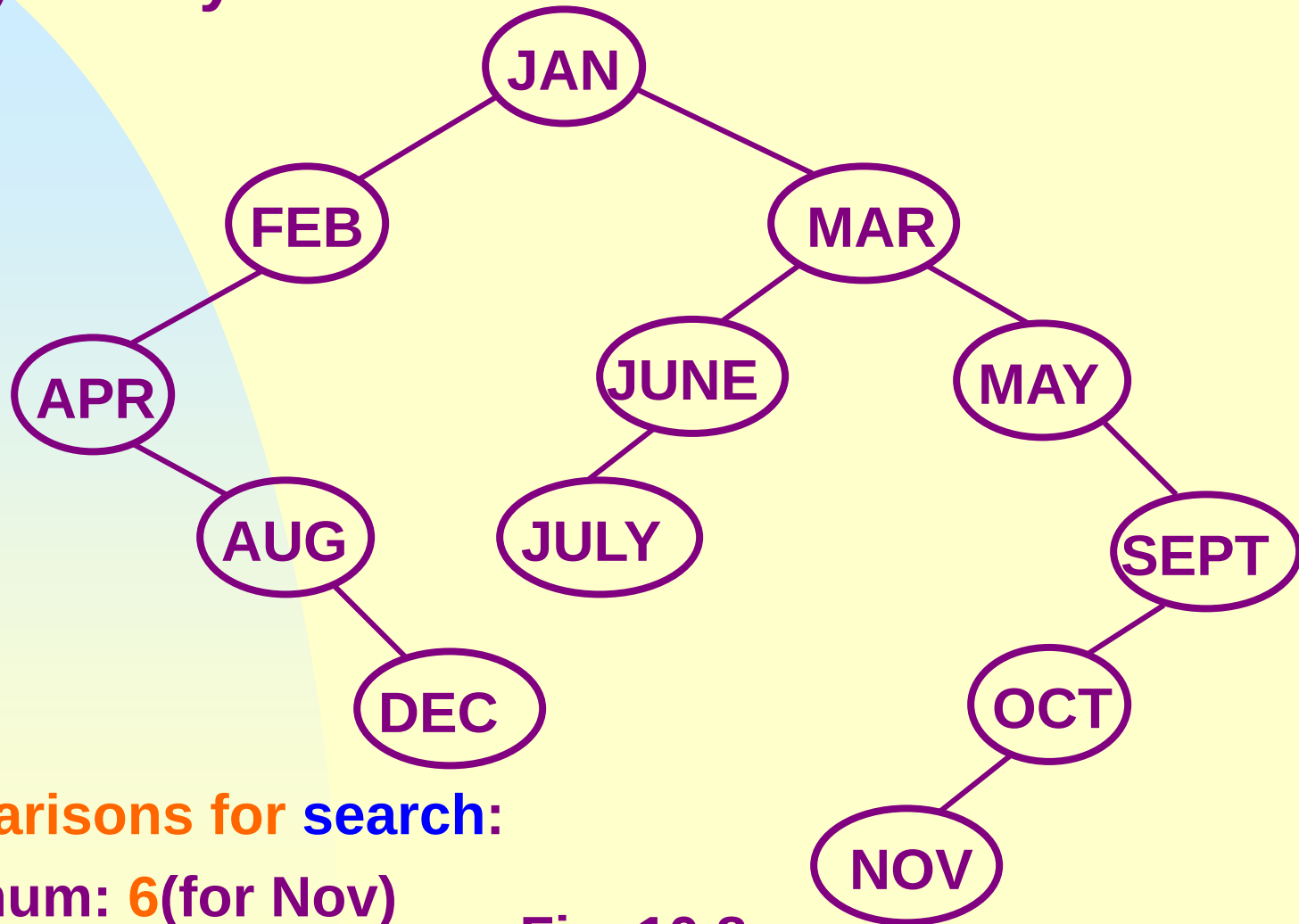
Chapter 10

Efficient Binary Search Trees

10.2 AVL Trees

At first, we show some **binary search trees**.

Fig. 10.8 shows the **binary search tree** obtained by inserting the months **JAN to DEC** into an initially empty binary search.



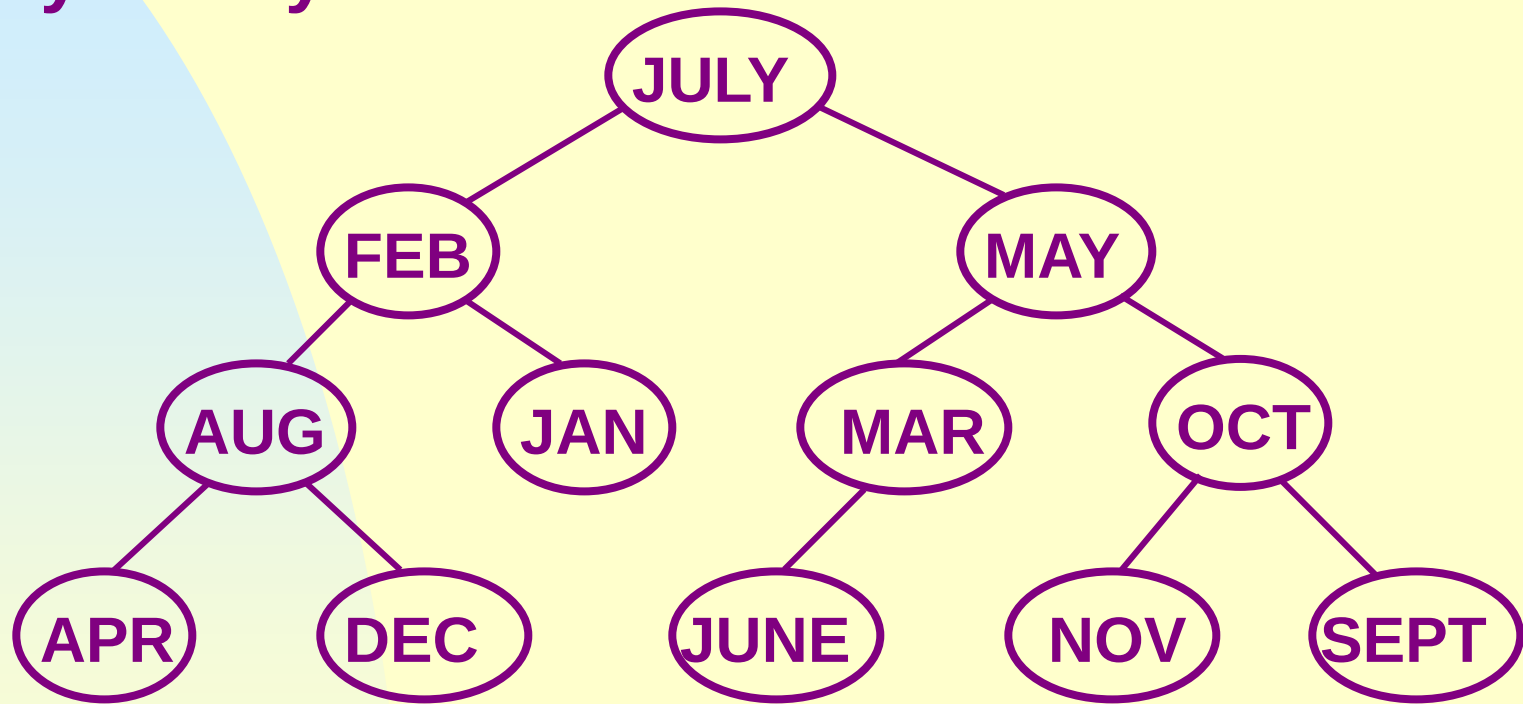
Comparisons for search:

Maximum: **6**(for Nov)

Average: $42/12 = \mathbf{3.5}$

Fig. 10.8

Fig. 10.9 shows the **binary search tree** obtained by inserting JULY, FEB, MAY, AUG, DEC, MAR, OCT, APR, JAN, JUNE, SEPT, NOV into an initially empty binary search tree.



Comparisons for **search**:

Maximum: **4**

Average: $37/12=$ **3.1**

Balanced

Fig. 10.9

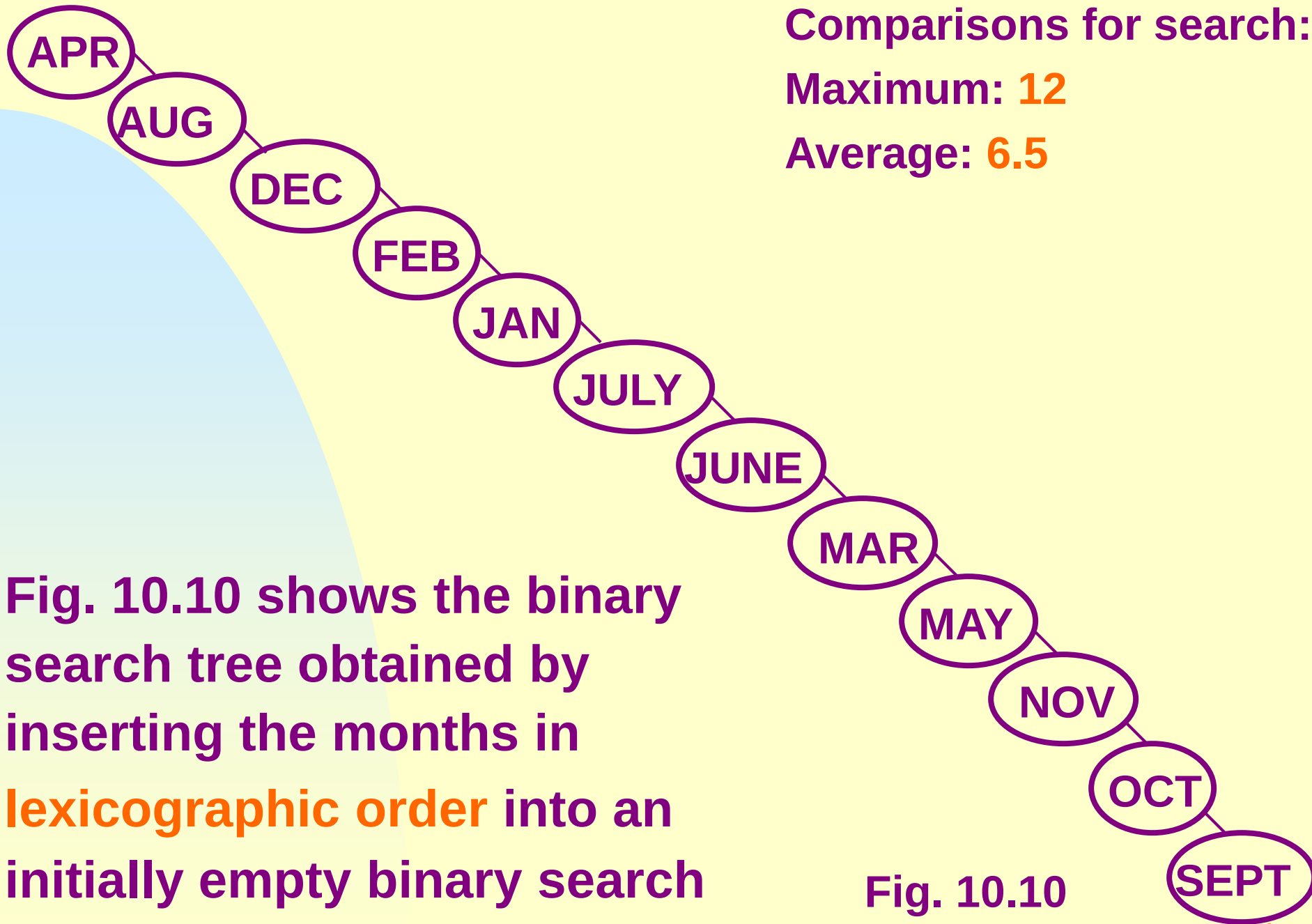


Fig. 10.10 shows the binary search tree obtained by inserting the months in **lexicographic order** into an initially empty binary search tree.

Some observations from the above BSTs:

- ✓ Obviously, both the **maximum and average search time** will be **minimized** if the **binary search tree (BST)** is maintained as a **complete binary tree (CBT)** at all times. The time complexity is $O(\log n)$ for a CBT with n nodes.
- ✓ However, due to dynamic conditions, achieving this ideal is **difficult** because it needs to **restructure** the whole tree in some cases.
- ✓ By maintaining a **balanced binary search tree (BST)** at all times, we can guarantee the maximum and average search time is also $O(\log n)$ if this BST containing n nodes.

AVL Tree

The **AVL** tree (introduced by Adelson-Velskii and Landis in 1962) is a **height-balanced binary search tree**.

Definition of height-balanced binary search tree (BST):

- ✓ An **empty tree** is an **height-balanced BST**.
- ✓ If T is a nonempty BST with T_L and T_R as its left and right subtrees respectively, then T is a **height-balanced BST** iff
 - (1) T_L and T_R are **height-balanced BST**, and
 - (2) $|h_L - h_R| \leq 1$ where h_L and h_R are the heights of T_L and T_R respectively.

Note: The definition of a height-balanced binary tree requires that **every subtree** also be height-balanced.

The BST in Fig. 10.8 is **not** height-balanced.

The BST in Fig. 10.9 is **height-balanced**.

The BST in Fig. 10.10 is **not** height-balanced.

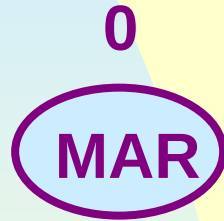
Definition: The **balance factor**, BF, of a node T in a binary tree is defined to be $h_L - h_R$.

For any node in an **AVL tree**, its BF is **-1, 0, or 1**.

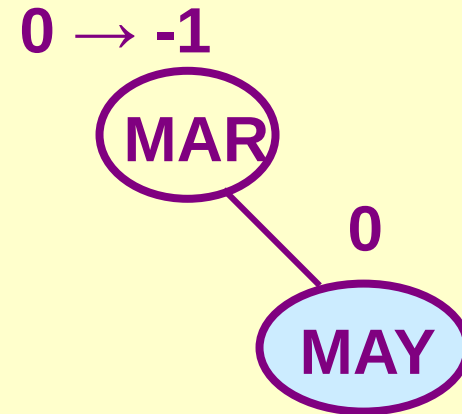
How to balance an unbalanced BST when inserting a node

- ✓ When a node is inserted into an AVL tree, the balance may be destroyed. If so, we should rebalance it.
- ✓ To rebalance an AVL tree, there are four possible rotation operations.
- ✓ We construct an AVL tree by inserting the months of the year to illustrate the processes for maintaining an AVL tree.
- ✓ The insertion order is: MAR, MAY, NOV, AUG, APR, JAN, DEC, JULY, FEB.

An example of constructing an AVL Tree via insertion



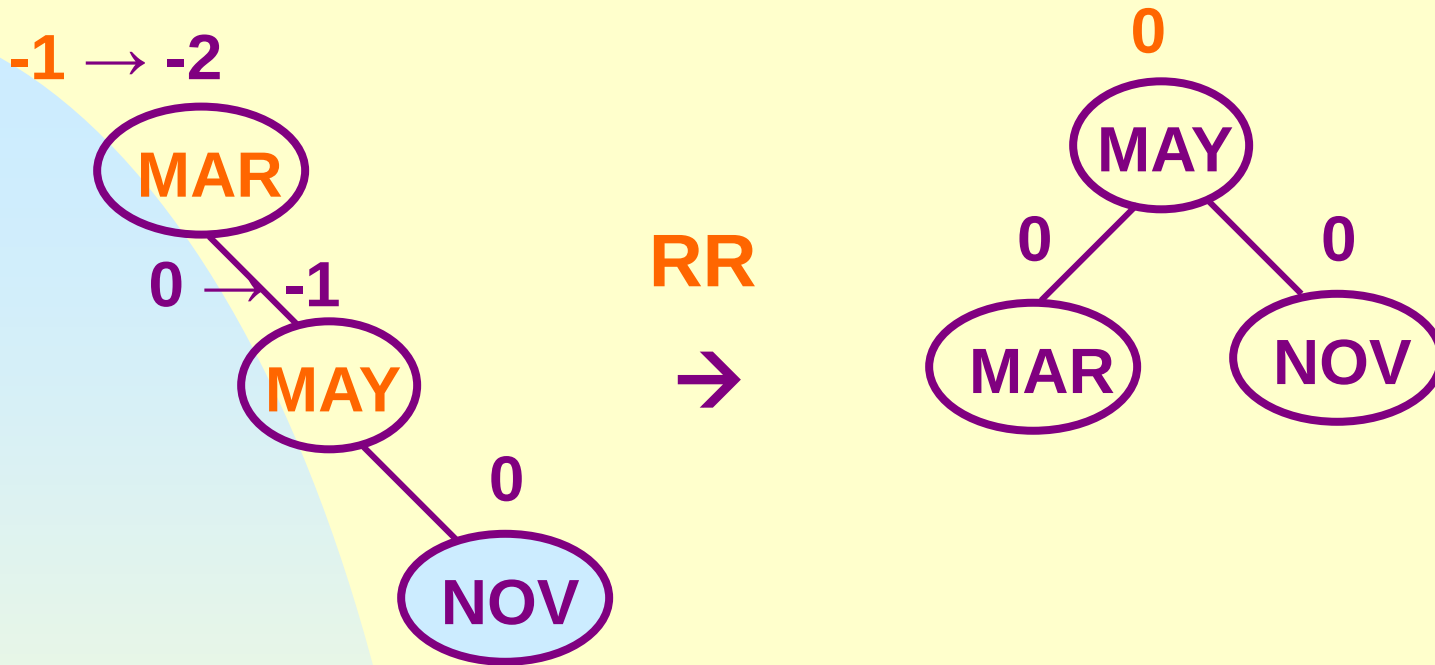
(a) Insert MAR



(b) Insert MAY

Figure 10.11

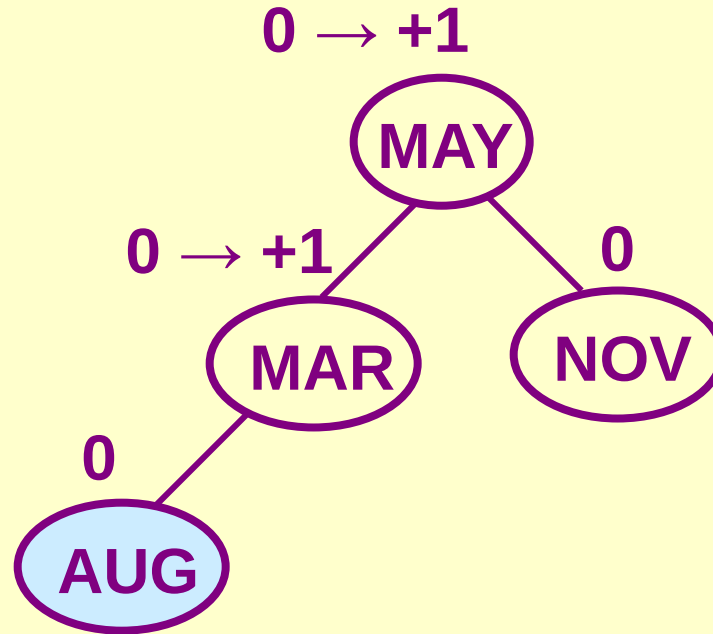
Insertion order: **MAR**, **MAY**, NOV, AUG, APR, JAN, DEC, JULY, FEB.



(c) Insert NOV

Figure 10.11

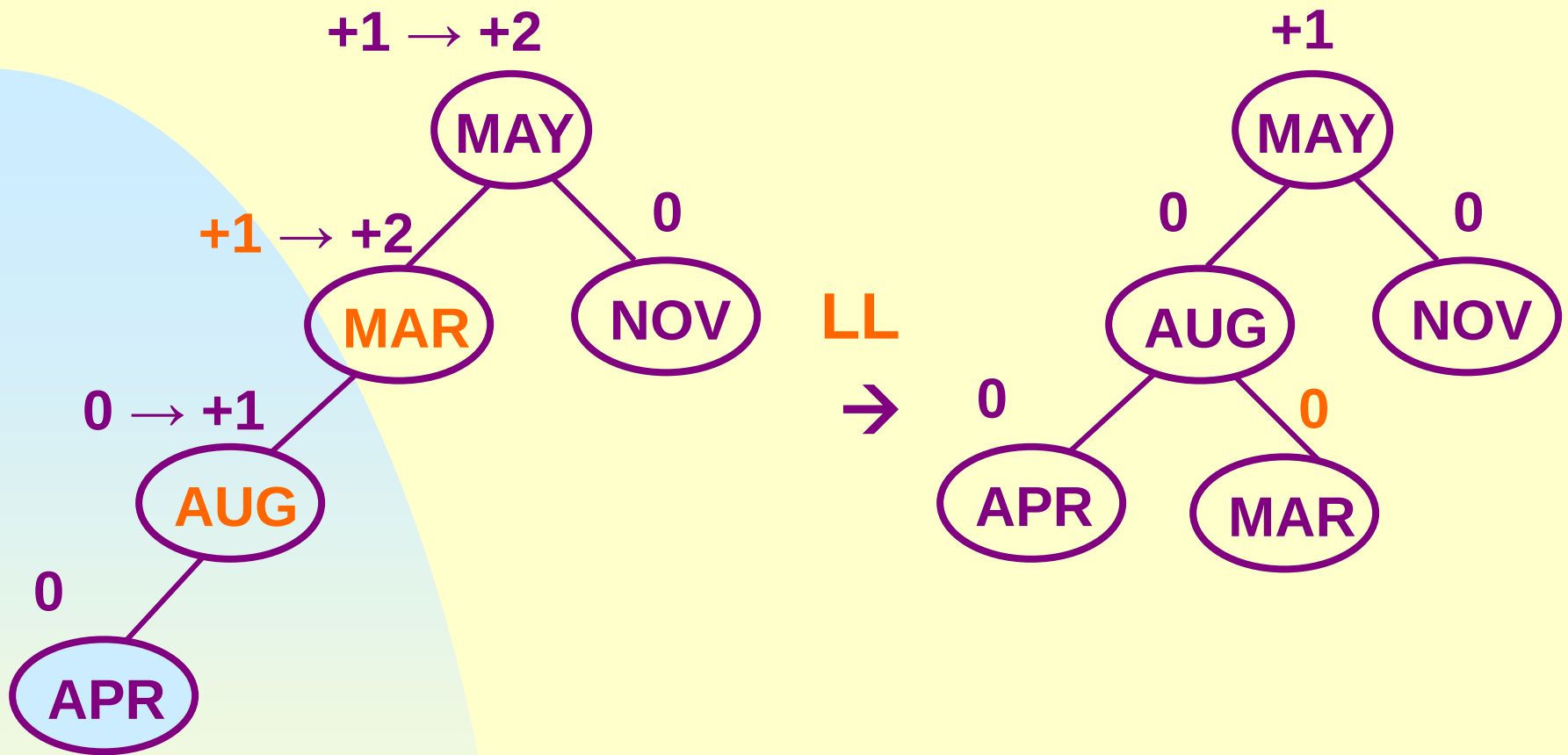
Insertion order: MAR, MAY, **NOV**, AUG, APR, JAN, DEC, JULY, FEB.



(d) Insert AUG

Figure 10.11

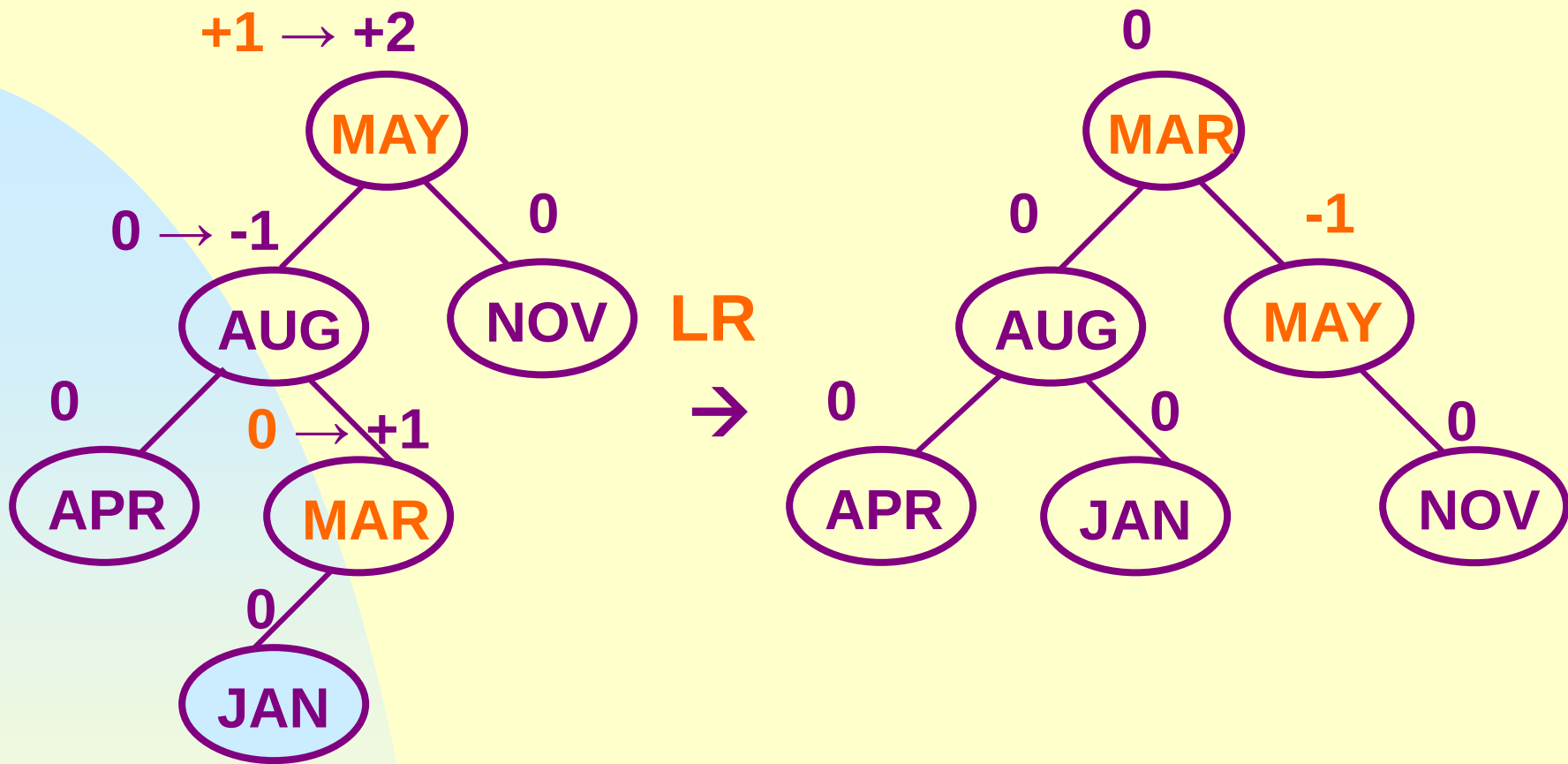
Insertion order: MAR, MAY, NOV, **AUG**, APR, JAN, DEC, JULY, FEB.



(e) Insert APR

Figure 10.11

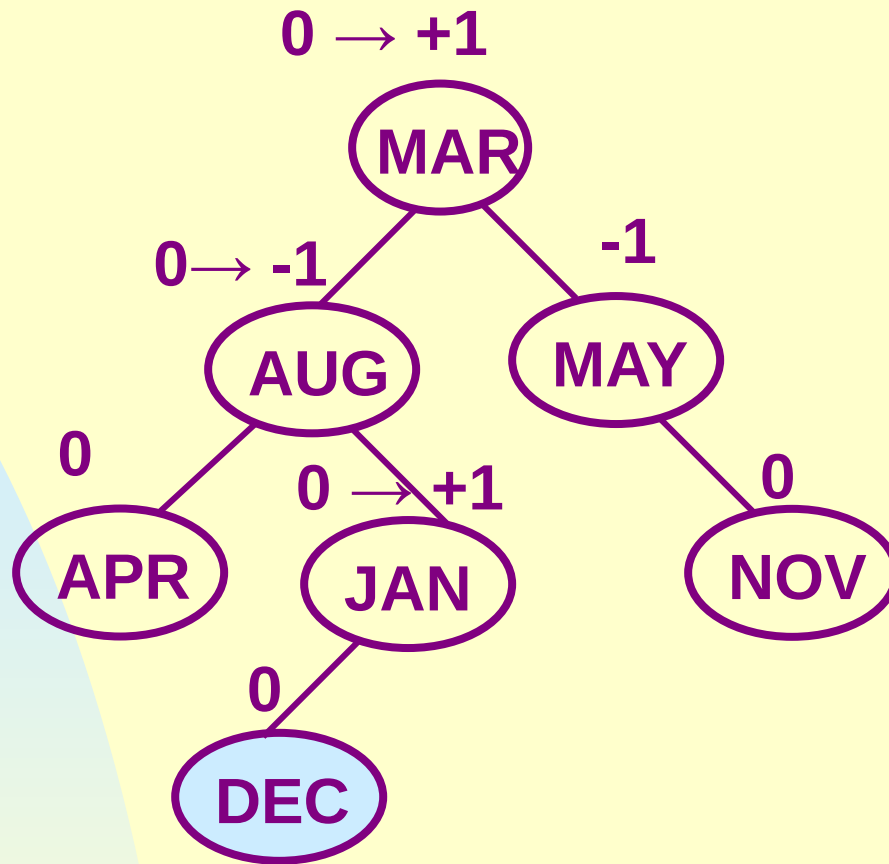
Insertion order: MAR, MAY, NOV, AUG, **APR**, JAN, DEC, JULY, FEB.



(f) Insert JAN

Figure 10.11

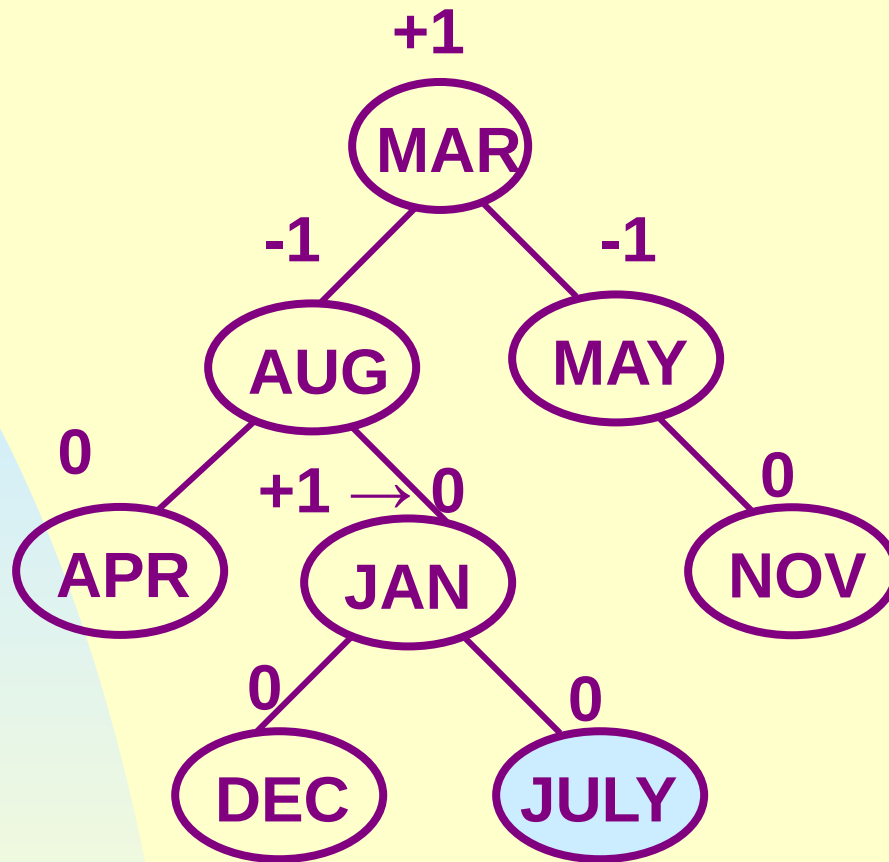
Insertion order: MAR, MAY, NOV, AUG, APR, **JAN**, DEC, JULY, FEB.



(g) Insert DEC

Figure 10.11

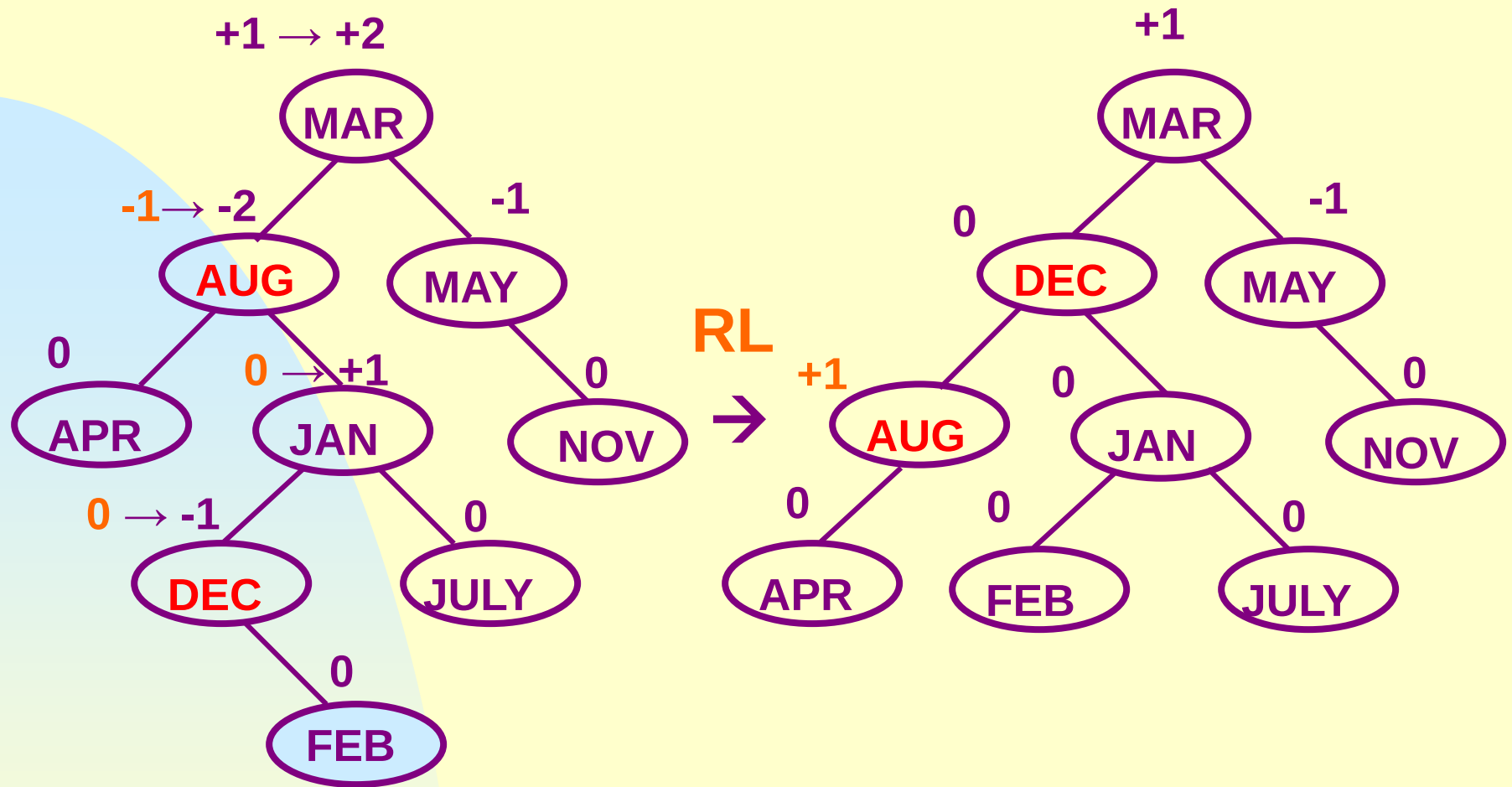
Insertion order: MAR, MAY, NOV, AUG, APR, JAN, **DEC**, JULY, FEB.



(h) Insert JULY

Figure 10.11

Insertion order: MAR, MAY, NOV, AUG, APR, JAN, DEC, **JULY**, FEB.



(i) Insert FEB

Figure 10.11

Insertion order: MAR, MAY, NOV, AUG, APR, JAN, DEC, JULY, **FEB**.

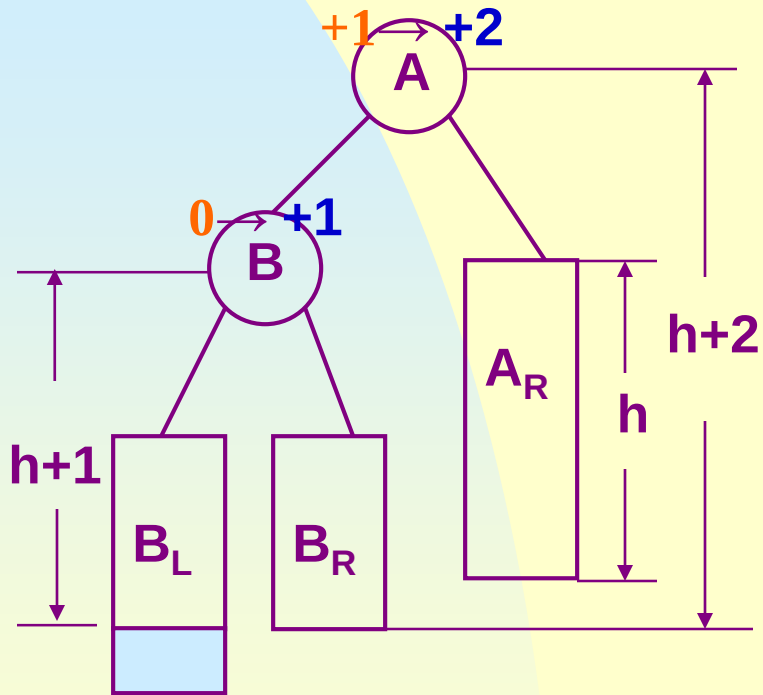
We can see:

- ✓ The **insertion of a node** to a balanced binary search tree **could unbalance** it.
- ✓ We use 4 different kinds of **rotations**: **LL**, **RR**, **LR**, and **RL** (Fig. (e), (c), (f) and (i), respectively) to rebalance the tree.
 - **LL**: Y is inserted in the **left** subtree of the **left** child of A.
 - **LR**: Y is inserted in the **right** subtree of the **left** child of A.
 - **RR**: Y is inserted in the **right** subtree of the **right** child of A.
 - **RL**: Y is inserted in the **left** subtree of the **right** child of A.

- ✓ LL and RR are symmetric, as are LR and RL.
- ✓ A rotation should be performed when the nearest ancestor of the newly inserted node becomes unbalanced (i.e., its BF changes to ± 2)—which had a balance factor (BF) of ± 1 before insertion.
- ✓ Now we give a more detailed analysis of four kinds of rotations.

LL Rotation

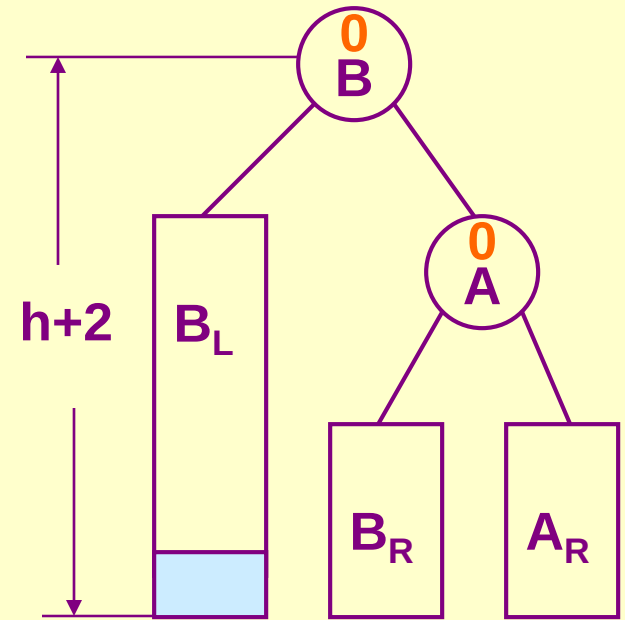
Unbalanced after
insertion



rotation type

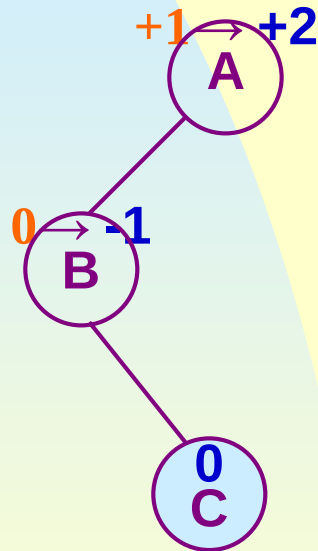
LL

Rebalanced



LR Rotation(a)

Unbalanced after
insertion

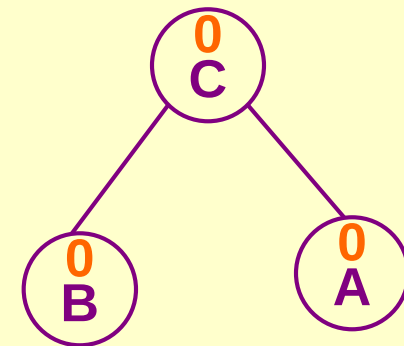


rotation type

LR(a)

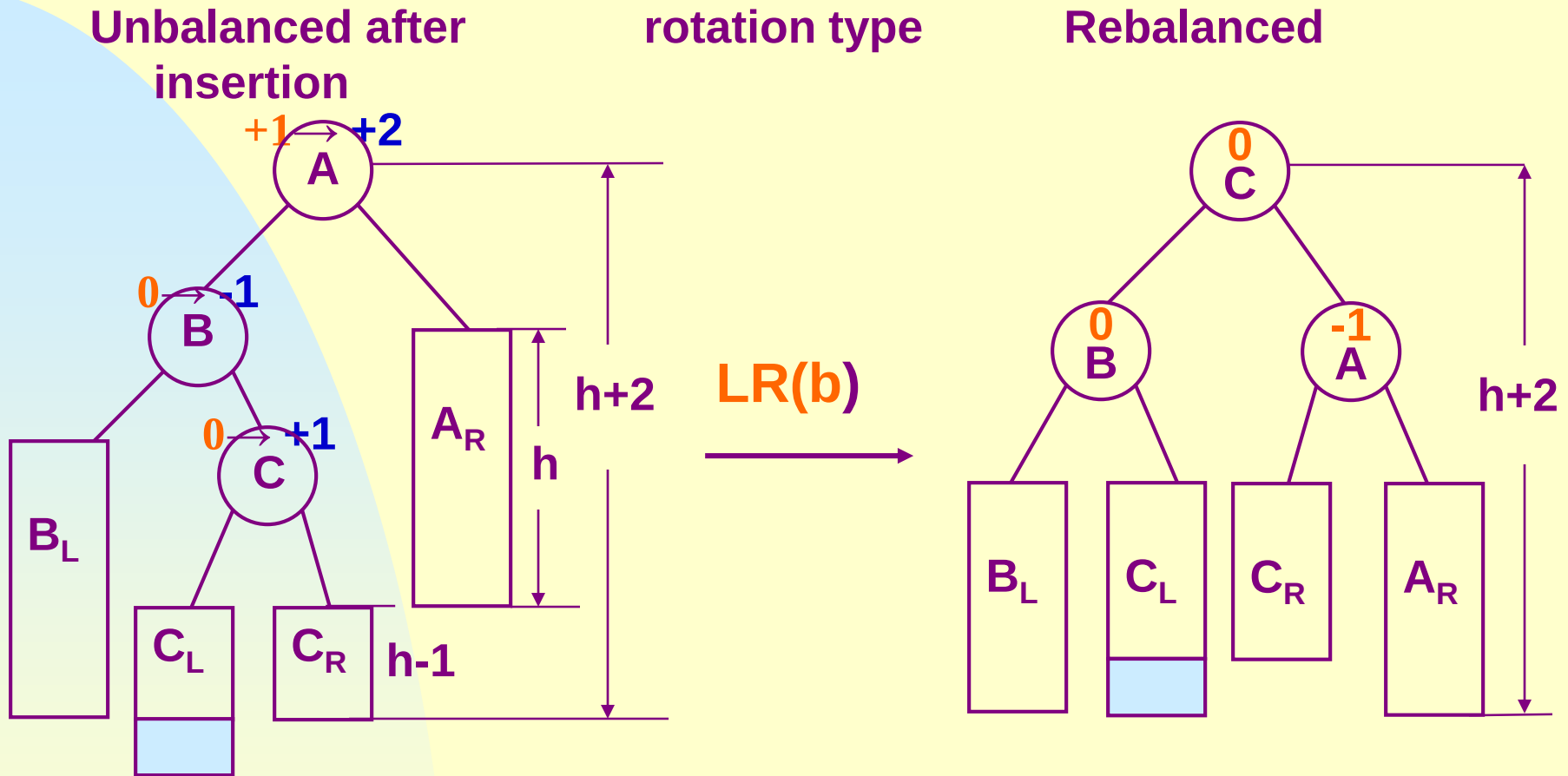


Rebalanced



Case a: Unbalanced after insertion, **BF of C is 0**.
C is the newly inserted node, A and B must have no other child, otherwise, this situation cannot happen.

LR Rotation(b)



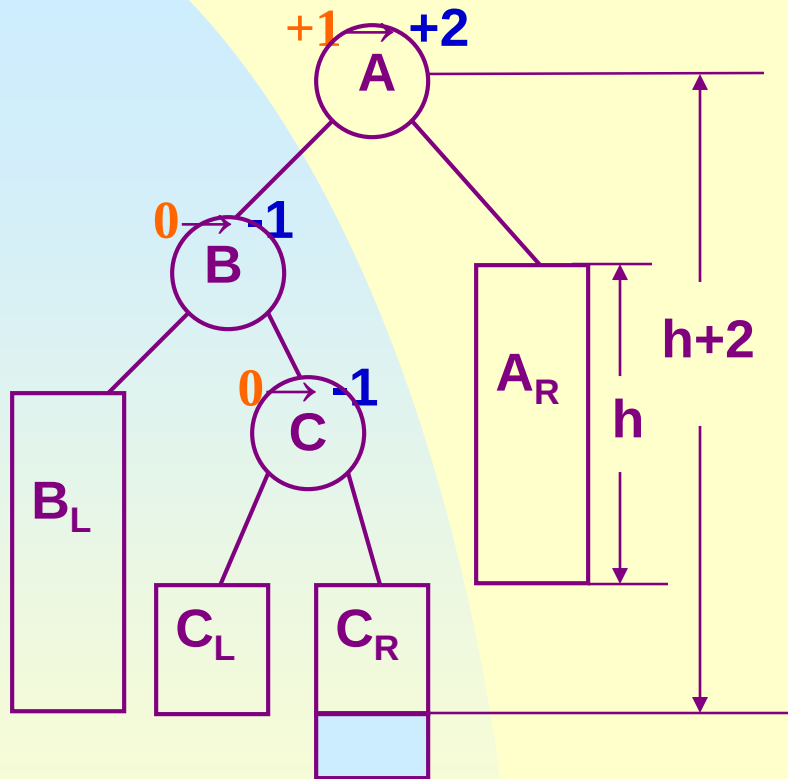
Case b: Unbalanced after insertion, BF of C is 1.
 The newly inserted node is inserted on the left
 subtree of C.

LR Rotation(c)

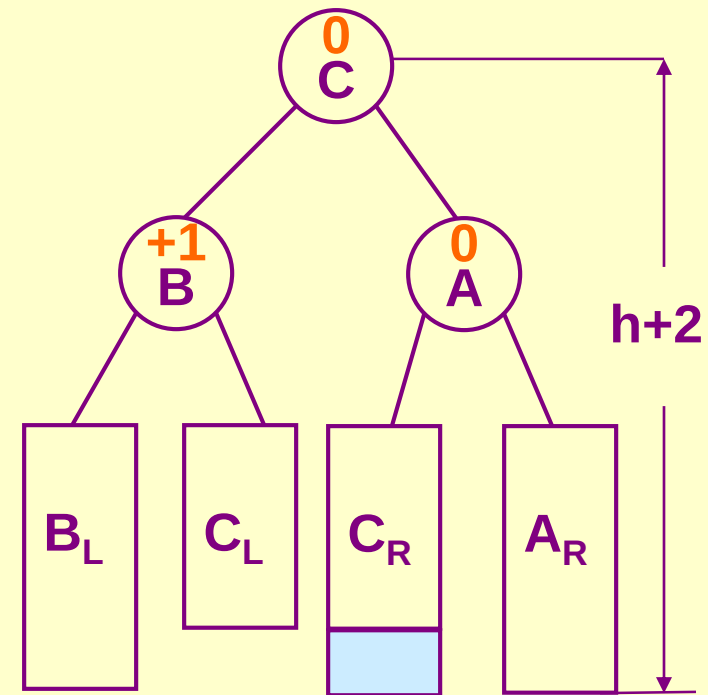
Unbalanced after insertion

rotation type

Rebalanced



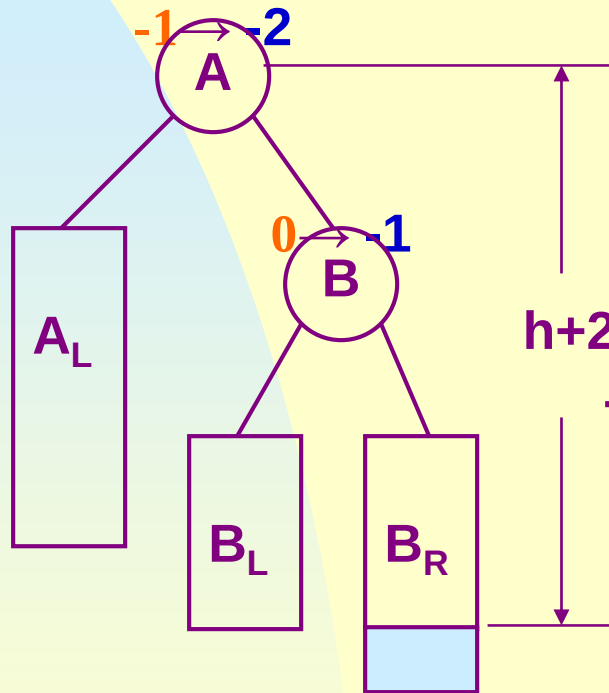
LR(c)



Case c: Unbalanced after insertion, BF of C is -1.
The newly inserted node is inserted on the right subtree of C.

RR Rotation

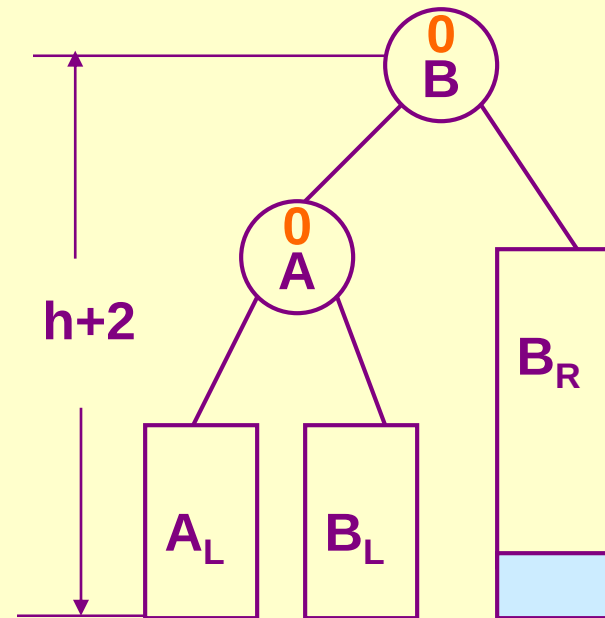
Unbalanced after
insertion



rotation type

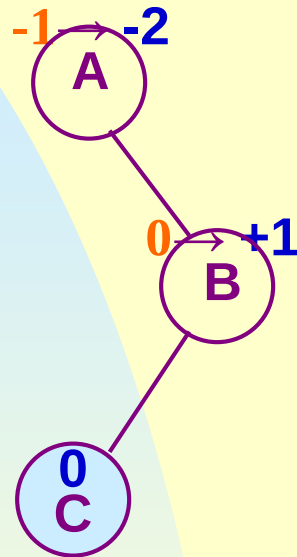
RR

Rebalanced



RL Rotation(a)

Unbalanced after
insertion

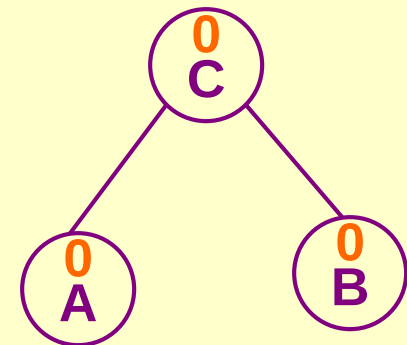


rotation type

RL(a)

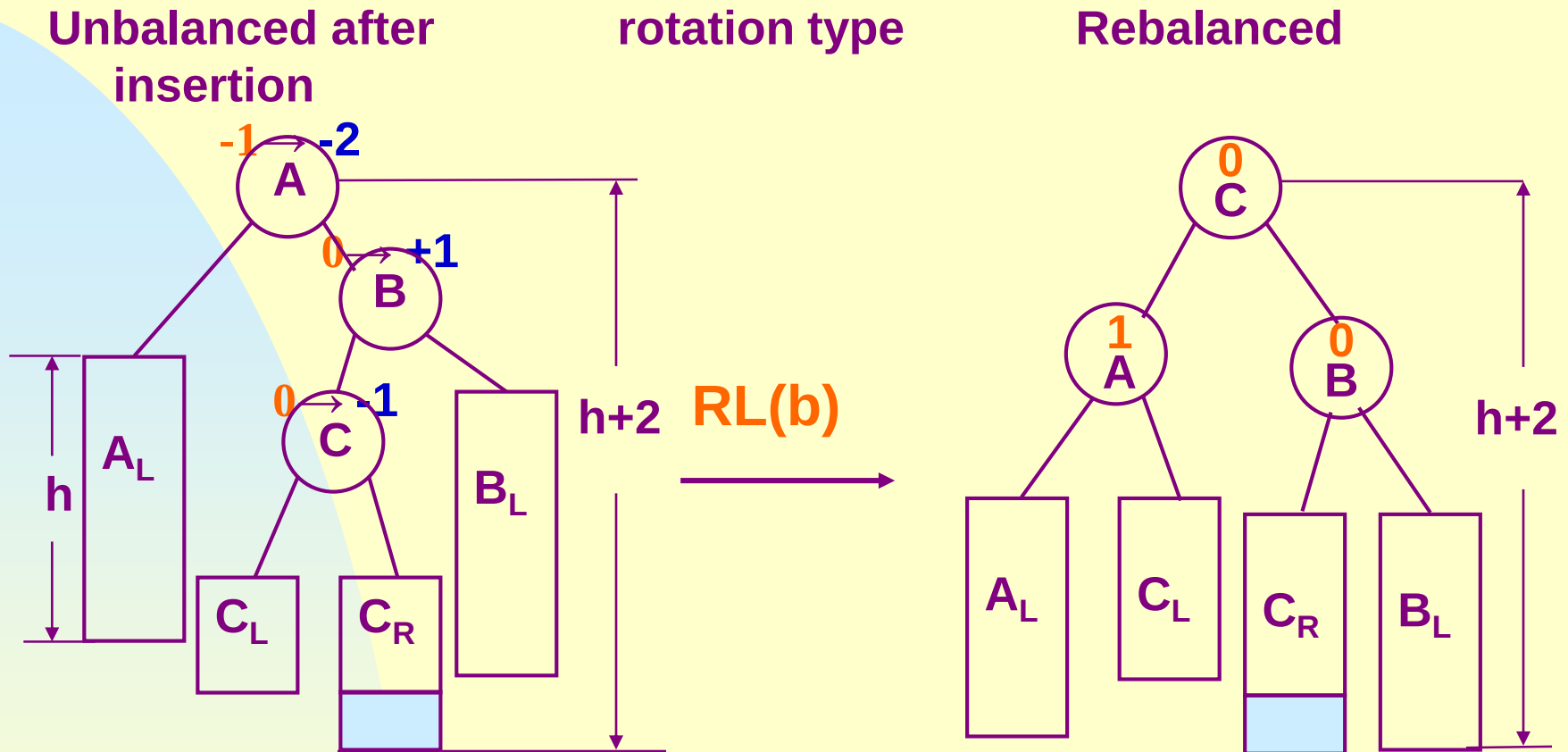


Rebalanced



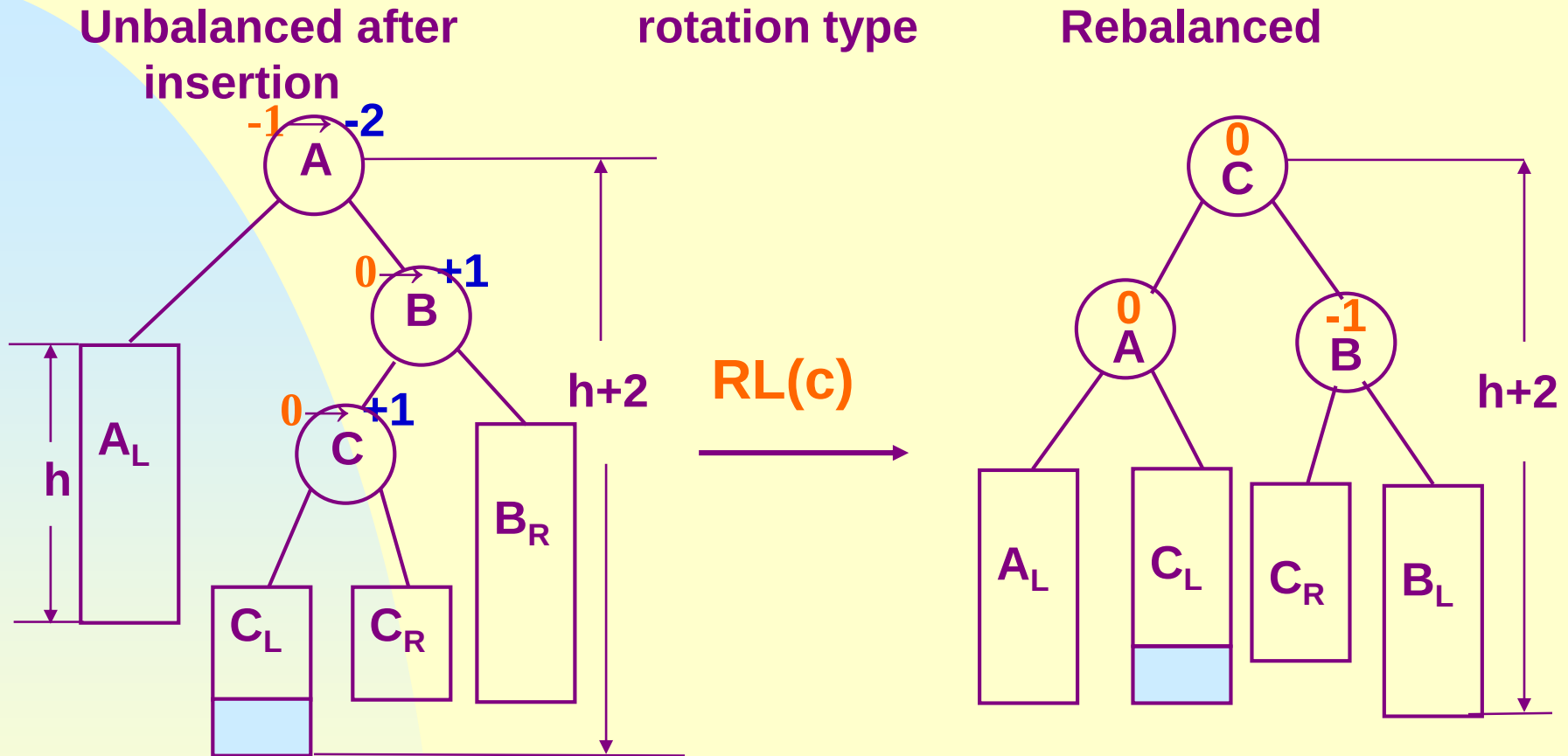
Case a: Unbalanced after insertion, BF of C is 0.
C is the newly inserted node, A and B must have no other child, otherwise, this situation cannot happen.

RL Rotation(b)



Case b: Unbalanced after insertion, BF of C is -1
 The newly inserted node is inserted on the right subtree of C.

RL Rotation(c)

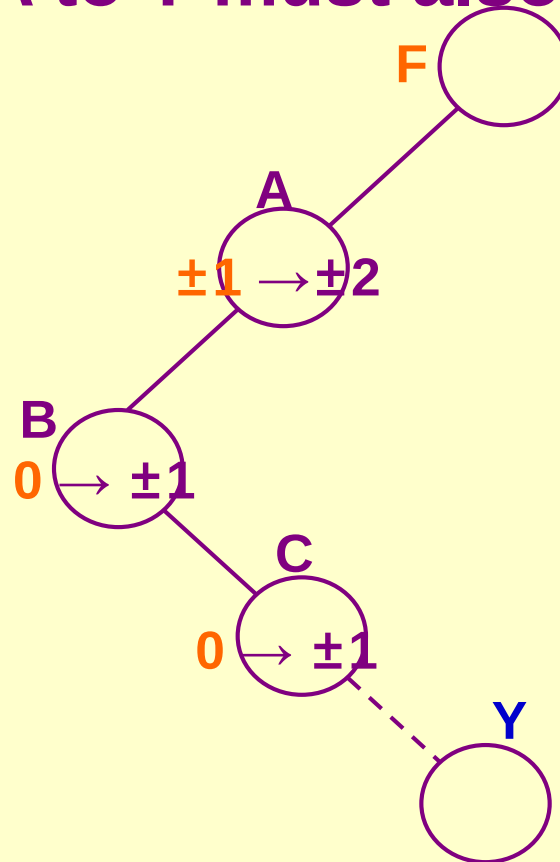


Case c: Unbalanced after insertion, BF of C is 1
 The newly inserted node is inserted on the left subtree of C.

Some observations about insertion

- To insert a new node to an AVL tree, we first search from the root to find the insertion point.
- (1) If there is a node A that is **the nearest ancestor** of the new inserted node Y with $BF = \pm 1$ **before insertion**, then we know:
 - **The original BFs of all nodes on the path from A (excluding A) to the new inserted node's parent must be 0.** Thus, after insertion, these nodes will remain balanced and their new BFs will be ± 1 .
 - **If the new node is inserted in the lower subtree of A**, then A will be still balanced and its new BF will be 0. The height of the subtree rooted at A will remain unchanged.

- If the new node is inserted in the higher subtree of A, then A will become an unbalanced node. A rotation should be carried out.
- To carry out the rotation, the parent node F of A, its child node B and its grandchild node C on the path from A to Y must also be known.



- After rebalancing, the height of the subtree involved in the rotation is the same as it is before insertion. So, it is unnecessary to examine the ancestors of A any more.

■ (2) If there is no node with $BF = \pm 1$ on the path from the root to the new inserted node Y before insertion. That is, the BFs of the nodes on the path from the root to the new inserted node's parent are all 0 before insertion. Then:

- The tree will remain balanced, and the BFs of the nodes on this path will be $= \pm 1$.
- After insertion, the height of the whole tree increase by one.

Further discussion for implementing the insertion algorithm

- As discussed above, it is necessary to find the “A” node.
- To find this node, **we initially let A be the root.** When there is a new node whose BF is ± 1 on the search path from the root to the insertion point, we update A to this node. We can see:
 - Whenever the insertion leads to a balanced or an unbalanced tree. The BF's of all nodes on the path from A (excluding A) to the parent of the new node Y must be 0 **before insertion**, and thus the BF's of these nodes will change to ± 1 **after insertion.**

- If the BF of A is 0 before insertion, that is, A is the root and the BF of the root is 0, then after insertion, it's BF is ± 1 , and the tree rooted at A is balanced.
- If the BF of A is ± 1 before insertion, and the new node is inserted in the lower subtree of A, then it's BF is 0, the subtree rooted at A is balanced, and the height of the subtree rooted at A does not change.
- If the BF of A is ± 1 before insertion, but the new node is inserted in the higher subtree of A, the subtree rooted at A is unbalanced.
- To carry out the rotation, the parent node F of A, its child node B and its grandchild node C on the path from A to Y must also be known.

The class definitions for AvlNode and AVL

```
template <class K, class E> class AVL;

template <class K, class E>
class AvlNode {
friend class AVL<K, E>;
public:
    AvlNode(const K& k, const E& e)
    {key=k; element=e; bf=0; leftChild=rightChild=0;}
private:
    K key;
    E element
    int bf;
    AvlNode<K, E> *leftChild, *rightChild;
};
```



```
template <class K, class E>
class AVL {
public:
    AVL(): root(0) { };
    E& Search(const K&) const;
    void Insert(const K&, const E&);
    void Delete(const K&);
private:
    AvlNode<K, E> *root;
};
```

Program 10.3: Insertion into an AVL tree

```
template <class K, class E>
void AVL<K, E>::Insert(const K& k, const E& e)
{
    if (!root) { // empty tree
        root=new AvlNode<K, E>(k, e);
        return;
    }
    // phase 1: Locate insertion point for e and A.
    AvlNode<K, E> *a=root, // a is the most recent node with BF  $\pm 1$ 
        *pa=0, // parent of a
        *p=root, // p moves through the tree
        *pp=0; // parent of p
```

```
while (p) { // search for insertion point for e and locate a
    if (p→bf) {a=p; pa=pp;}
    if (k<p→key) {pp=p; p=p→leftChild;}
    else if (k>p→key) {pp=p; p=p→rightChild;}
    else {p→element=e; return;} // k already in the tree
} // end of while
```

// phase 2: Insert and rebalance. k is not in the tree and
// will be inserted as the appropriate child of pp.

```
AvlNode<K, E> *y=new AvlNode<K, E>(k, e);
if (k<pp→key) pp→leftChild=y;    // as left child
else pp→rightChild=y;           // as right child
```

```
int d;
```

```
//d=+1, k is inserted in the left subtree of a; d=-1 in the right.
```

```
AvlNode<K, E> *b, // child of a
```

```
        *c; // child of b
```

```
if (k>a→key) { b=p=a→rightChild; d=-1;}
```

```
else { b=p=a→leftChild; d=1;}
```

```
// Adjust BF's of nodes on path from a to pp
```

```
while (p!=y) //nodes on a to y have BF's of 0 before insertion
```

```
    if (k>p→key) { // height of right increases by 1
```

```
        p→bf= -1; p=p→rightChild;
```

```
    }
```

```
    else { // height of left increases by 1
```

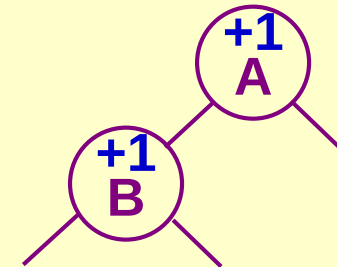
```
        p→bf= 1; p=p→leftChild;
```

```
    }
```

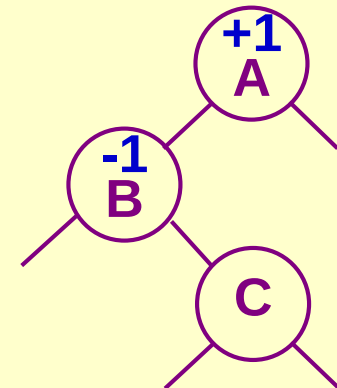
```
// Is tree unbalanced? if a's bf is 0, a is the root
if (!(a→bf) || !(a→bf +d)) { // tree still balanced
    a→bf +=d; return;
}
```

//tree unbalanced, determine rotation type

```
if (d==1) { // left imbalance, inserted in left subtree of a
    if (b→bf==1) { // type LL, inserted in left subtree of b
        a→leftChild=b→rightChild;
        b→rightChild=a; a→bf=0; b→bf=0;
    } //end of LL
```



```
else { // type LR, b→bf==-1, inserted in right subtree of b
    c=b→rightChild;
    b→rightChild=c→leftChild;
    a→leftChild=c→rightChild;
    c→leftChild=b;
    c→rightChild=a;
```



```

switch (c→bf) {
    case 1: // LR(b)
        a→bf=-1; b→bf=0;
        break;
    case -1: // LR(c)
        a→bf=0; b→bf=1;
        break;
    case 0: // LR(a)
        a→bf=0; b→bf=0;
        break;
} // end of switch
c→bf=0; b=c;
// Let b the root of the tree rotated.
} // end of LR

```

Before LR	After LR
a→bf=2	a→bf=-1
b→bf=-1	b→bf=0
c->bf=1	c->bf=0

Before LR	After LR
a→bf=2	a→bf=0
b→bf=-1	b→bf=1
c->bf=-1	c->bf=0

Before LR	After LR
a→bf=2	a→bf=0
b→bf=-1	b→bf=0
c->bf=0	c->bf=0

```

a->bf = (-c->cf - |c->cf|)/2;
b->bf = (-c->cf + |c->cf|)/2;
c->bf=0;

```

```
} // end of left imbalance
```

```
else { // right imbalance: symmetric to left imbalance  
    codes are in the next two slides.
```

```
}
```

```
// Subtree with root b has been rebalanced.
```

```
if (!pa) root=b; //before rotation, a is the root
```

```
else if (a==pa→leftChild)//a is pa's left child before insertion
```

```
    pa→leftChild=b;
```

```
    else pa→rightChild=b;
```

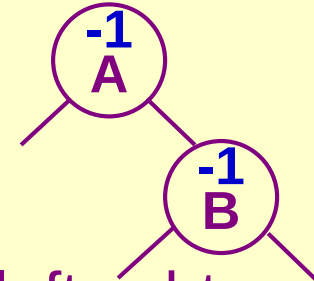
```
return;
```

```
} // end of AVL::Insert
```

```

else { //d=-1, right imbalance, inserted in right subtree of a
  if (b→bf==-1) { // type RR, inserted in right subtree of b
    a→rightChild=b→leftChild;
    b→leftChild=a; a→bf=0; b→bf=0;
  } //end of RR

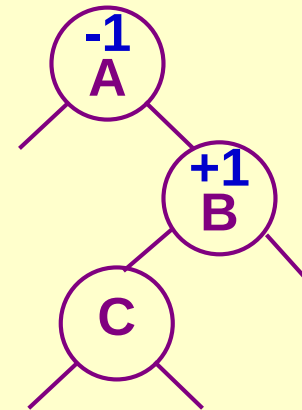
```



```

else { // type RL, b→bf==1, inserted in left subtree of b
  c=b→leftChild;
  b→leftChild=c→rightChild;
  a→rightChild=c→leftChild;
  c→rightChild=b;
  c→leftChild=a;

```




```

switch (c→bf) {
    case -1: // RL(b)
        a→bf=1; b→bf=0;
        break;
    case 1: // RL(c)
        a→bf=0; b→bf=-1;
        break;
    case 0: // RL(a)
        a→bf=0; b→bf=0;
        break;
} // end of switch
c→bf=0; b=c;
    // Let b the root of the tree rotated.
} // end of RL

```

Before RL	After RL
a→bf=-2	a→bf=1
b→bf=1	b→bf=0
c->bf=-1	c->bf=0

Before RL	After RL
a→bf=-2	a→bf=0
b→bf=1	b→bf=-1
c->bf=1	c->bf=0

Before RL	After RL
a→bf=-2	a→bf=0
b→bf=1	b→bf=0
c->bf=0	c->bf=0

```

a->bf = (-c->cf + |c->cf|)/2;
b->bf = (-c->cf - |c->cf|)/2;
c->bf=0;

```

Analysis of insertion:

- ✓ If h is the height of AVL tree before insertion, then the time to insert a new key is $O(h)$.
- ✓ We will show an AVL tree with n nodes, then its height h is $O(\log n)$, so the insertion time is $O(\log n)$.
- ✓ To explain this, let N_h be the **minimum number of nodes** in an AVL tree of **height h** . In the worst case, the heights of its subtrees are $h-1$ and $h-2$, and they both are AVL trees. Hence

$$N_h = N_{h-1} + N_{h-2} + 1 \text{ (the number of root)} \quad \text{and}$$

$$N_0 = 0, \quad N_1 = 1, \quad N_2 = 2.$$

So, $(N_h + 1) = (N_{h-1} + 1) + (N_{h-2} + 1)$, we can see

$$N_h = F_{h+2} - 1 \text{ for } h \geq 0.$$

Where F_h the h -th Fibonacci number

$$F_h = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^h - \left(\frac{1 - \sqrt{5}}{2} \right)^h \right]$$

$$F_h \approx \phi^h / \sqrt{5} \quad \phi = (1 + \sqrt{5}) / 2$$

$$N_h \approx \phi^{h+2} / \sqrt{5} - 1$$

If there are n nodes in the AVL tree, then its height h is at most

$$\log_{\phi} \left(\sqrt{5} (n + 1) \right) - 2 \approx 1.44 \log (n + 1) - 0.33$$

Exercise

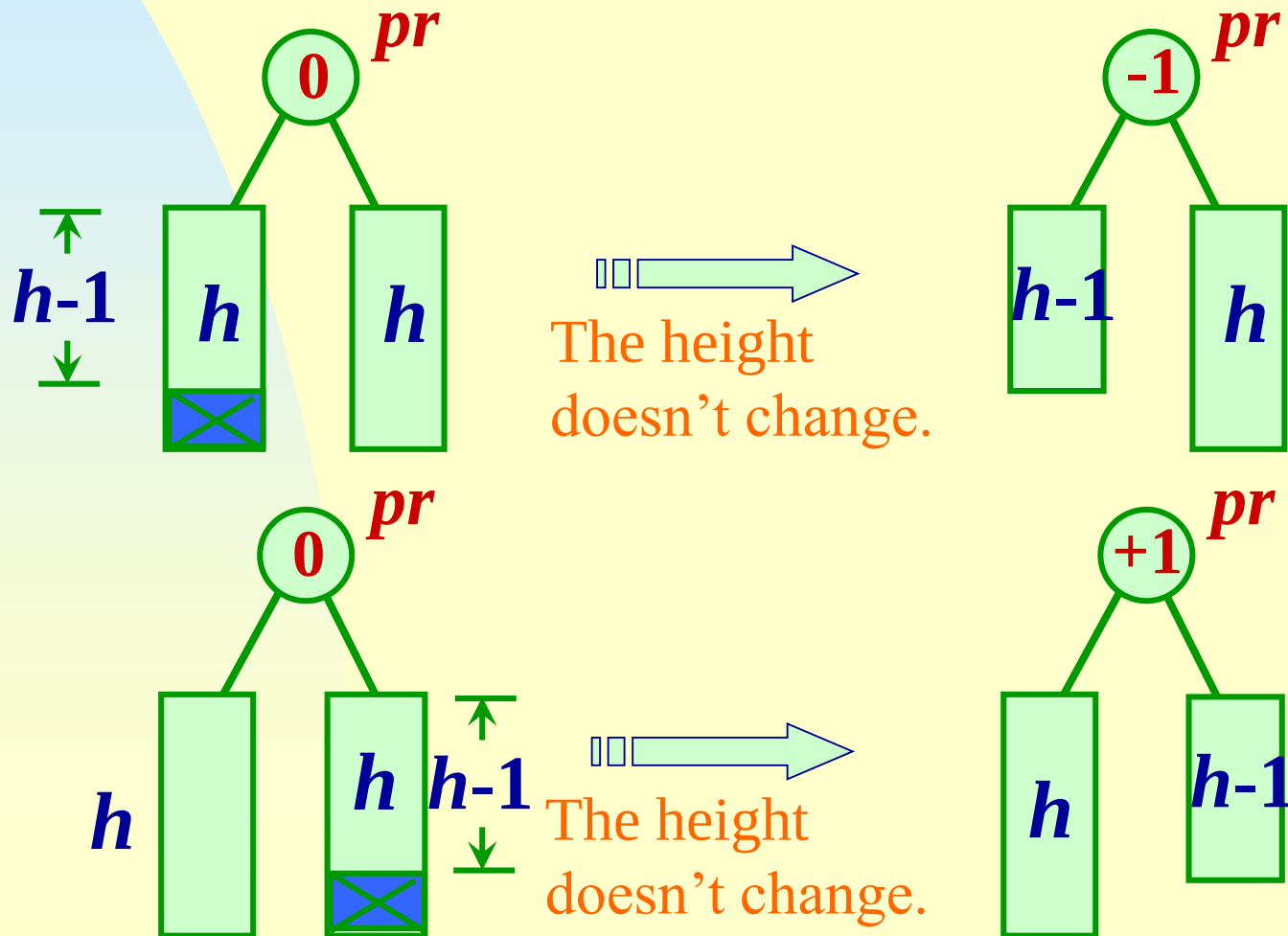
Question: 8 keys {1, 12, 3, 14, 18, 8, 6, 5} are inserted one by one into an initial empty AVL tree, please illustrate the step-by-step process of inserting these keys..

Delete nodes from a AVL Tree (self study)

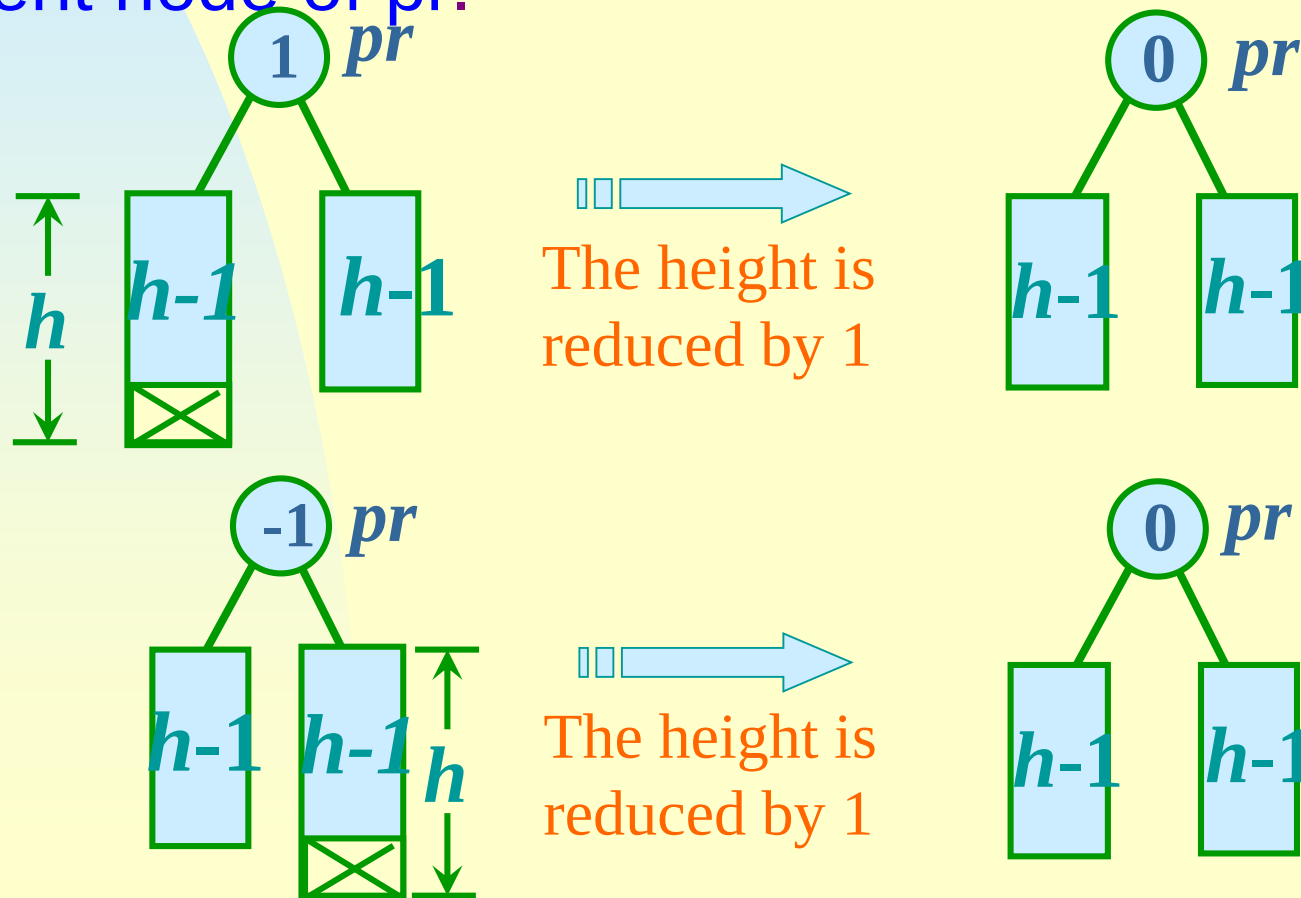
- 1. If the deleted node p has two children:
 - ✓ Search for the in-order immediate predecessor of p (or the in-order immediate successor of p). Let it be p' , which is a node with at most one child.
 - ✓ Replace the data of p with the data of node p' .
 - ✓ Now the node p' is regarded as the deleted node p .

- 2. If the deleted node p has at most one child s (may be NULL), it is simple:
 - ✓ We make the parent of p (let it be pr) be the parent of s , that is, s is the son of pr ;
 - ✓ The height of the subtree that originally rooted at p (now rooted at s) is reduced by 1, thus pr 's BF changes.
 - ✓ There are three cases:

(1) If the BF of pr is originally 0 before deletion of p , its new BF is ± 1 , the node pr is still balanced, and the height of the subtree rooted at pr does not change. The rebalancing process can be stopped.



(2) The BF of pr is originally ± 1 before deletion of p , and its higher subtree is shortened. Then the new BF of pr is 0. The subtree rooted at pr is balanced, but its height is reduced by 1. Therefore, it is necessary to examine the balance state of the parent node of pr .



(3) The BF of the node pr is originally ± 1 before deletion of p , and its shorter subtree was shortened, an imbalance occurs at the node pr . It needs to perform balance rotation to restore balance.

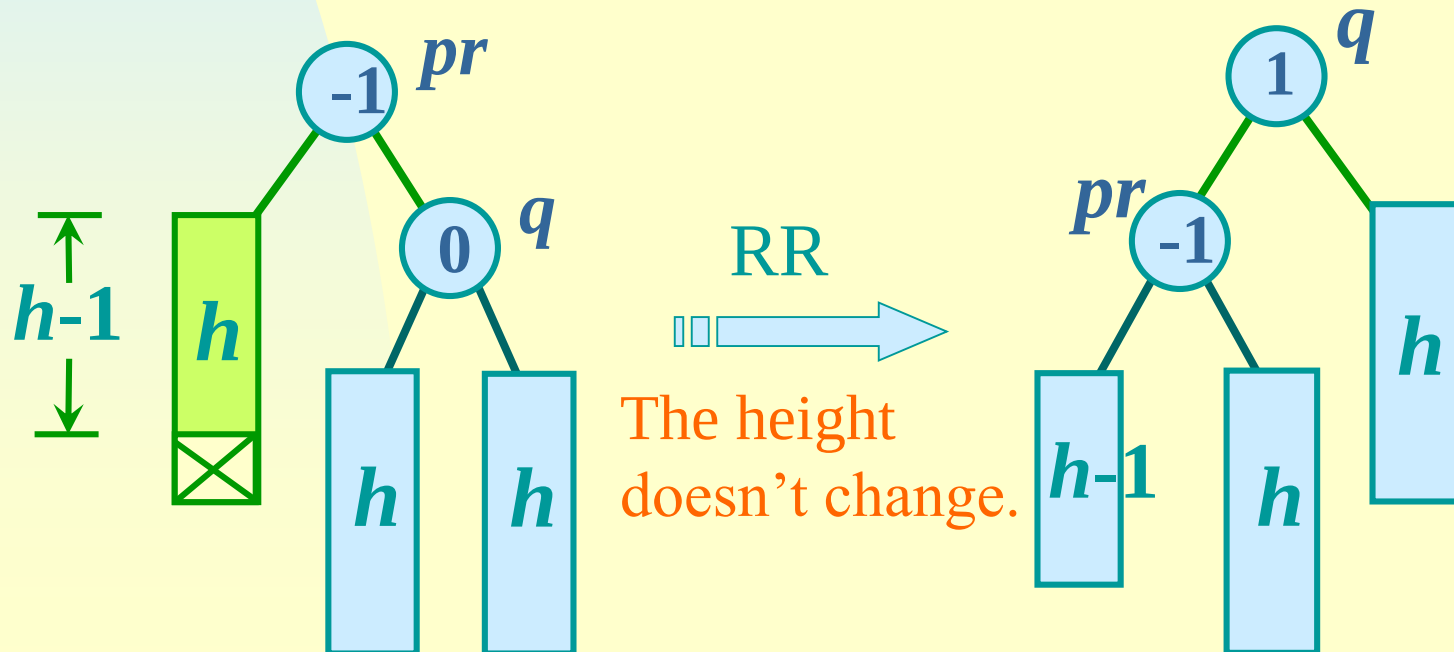
---To perform balance rotation, we need find the root of pr 's higher subtree q , and the root of q 's higher subtree r .

---According to the original BF of q (0 or ± 1), we consider three cases to balance:

a) If the original BF of q (the higher subtree of pr) is 0 before deletion, a single rotation is performed to restore balance.

--- pr 's BF is not changed but q 's BF is changed to the opposite number of pr 's BF.

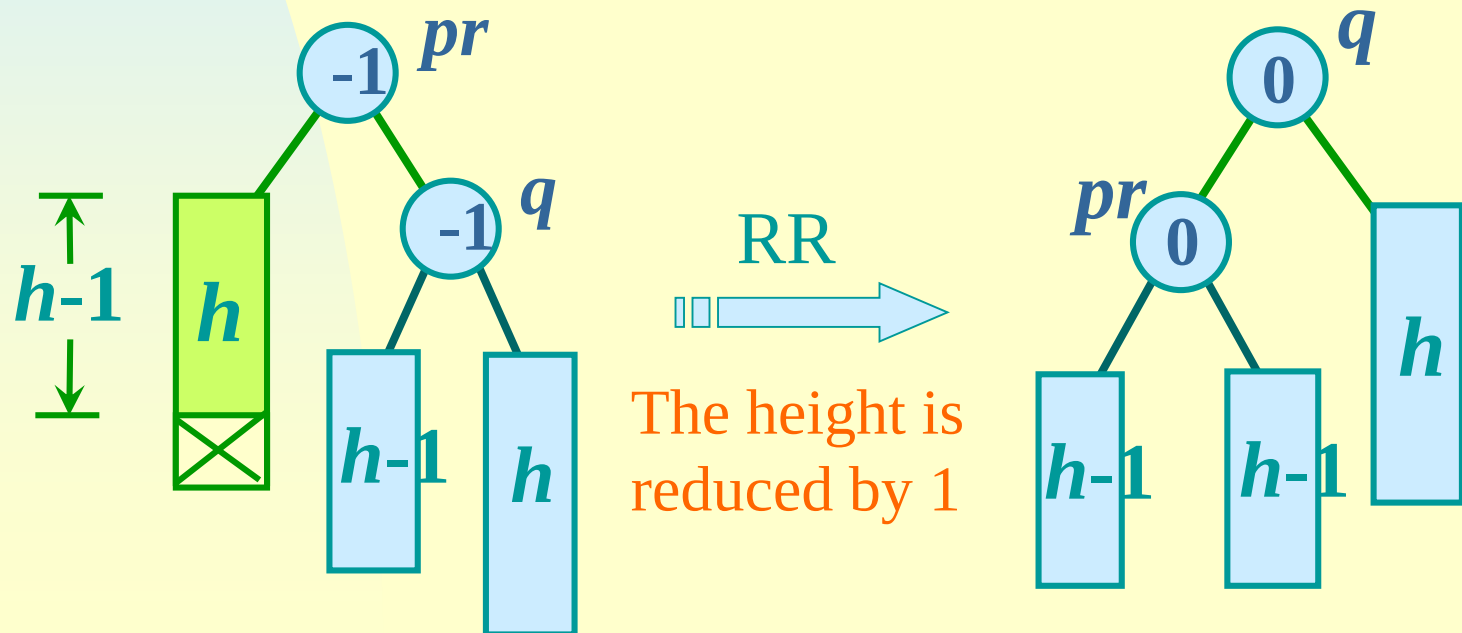
---Since the height of the new subtree rooted at q remains unchanged after the balance rotation, the rebalancing process can be stopped.



b) If the signs of q 's original BF and pr 's original BF are the same before deletion, a single rotation is performed to restore balance.

---The new pr 's BF and q 's BF are both 0.

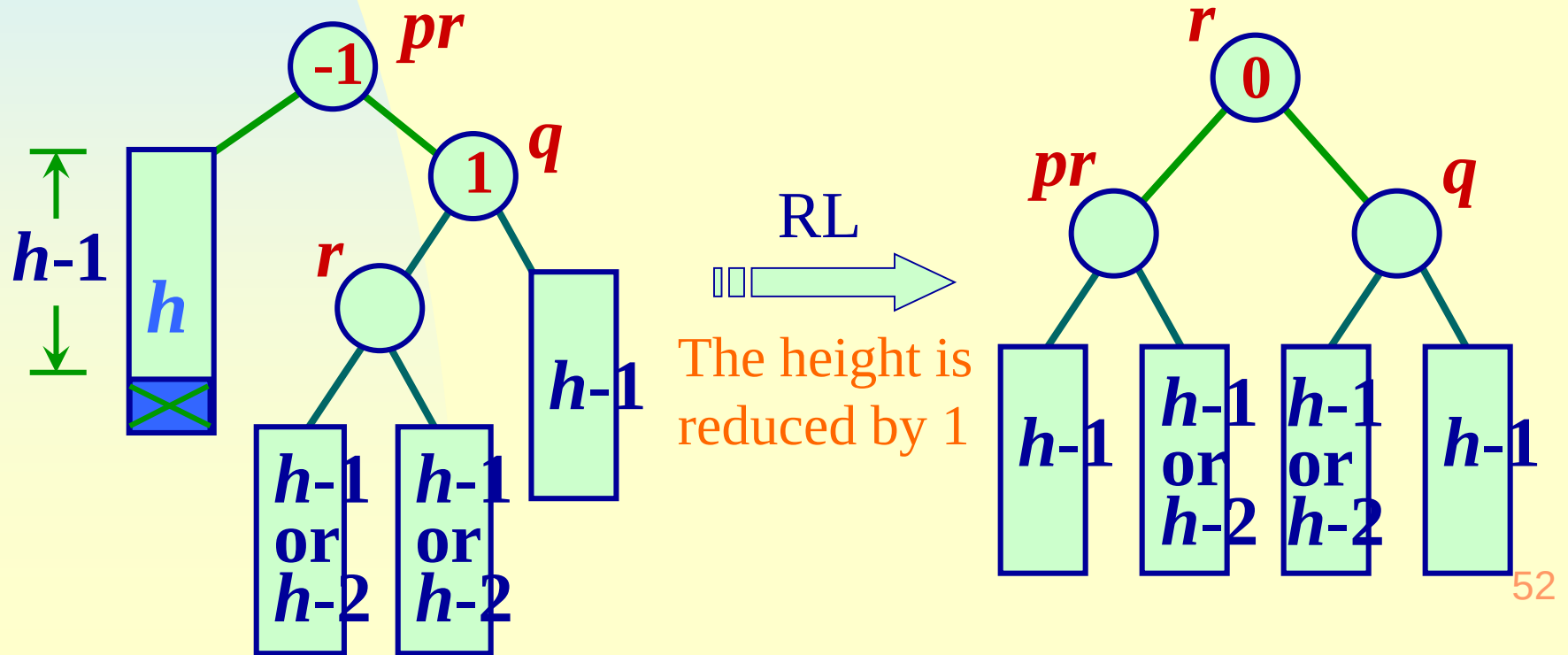
---Since the height of the subtree is reduced by 1 after the balance rotation, it is necessary to examine the balance states of nodes in upper levels.



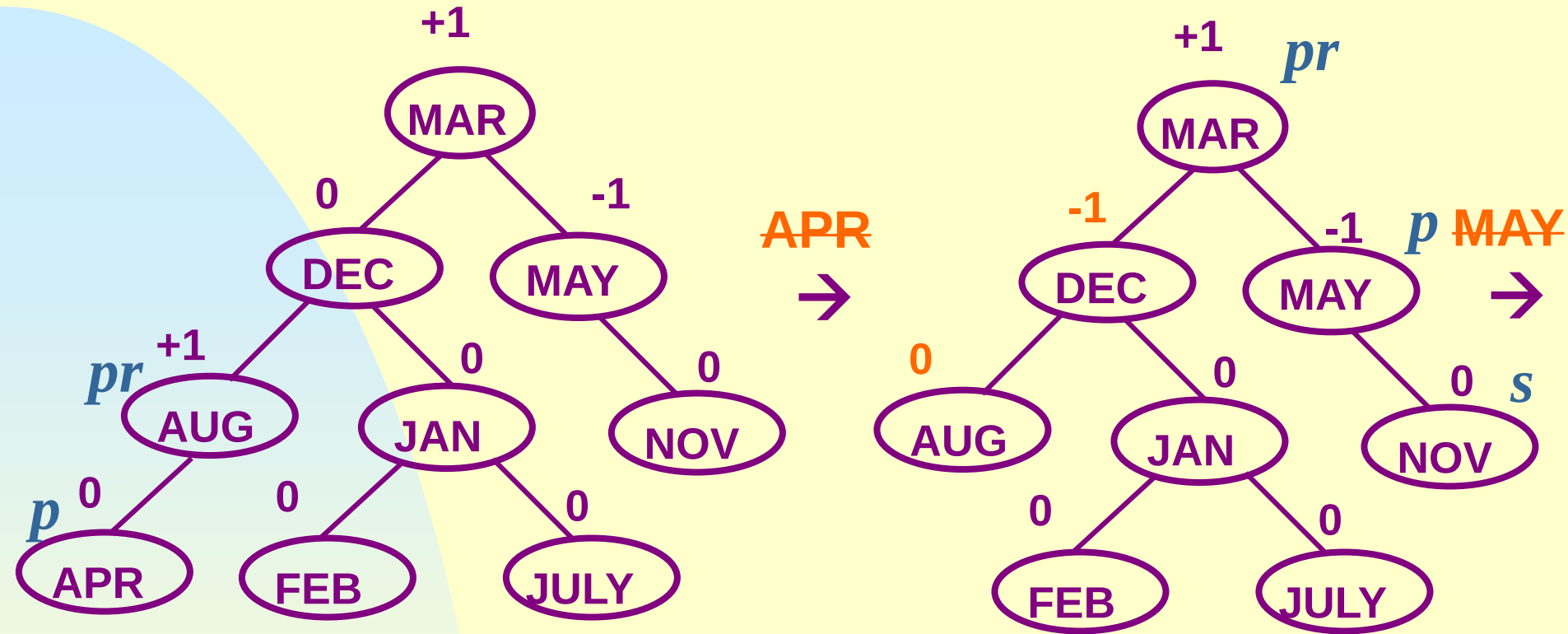
c) If the signs of q 's original BF and pr 's original BF are not the same before deletion, a double rotation is performed to restore balance.

---The new BF of r is set to 0, and pr 's new BF and q 's new BF can be determined by r 's original BF and the style of rotation.

---The height of the subtree is reduced by 1 after balancing. It is necessary to examine the balance states of nodes in upper levels.



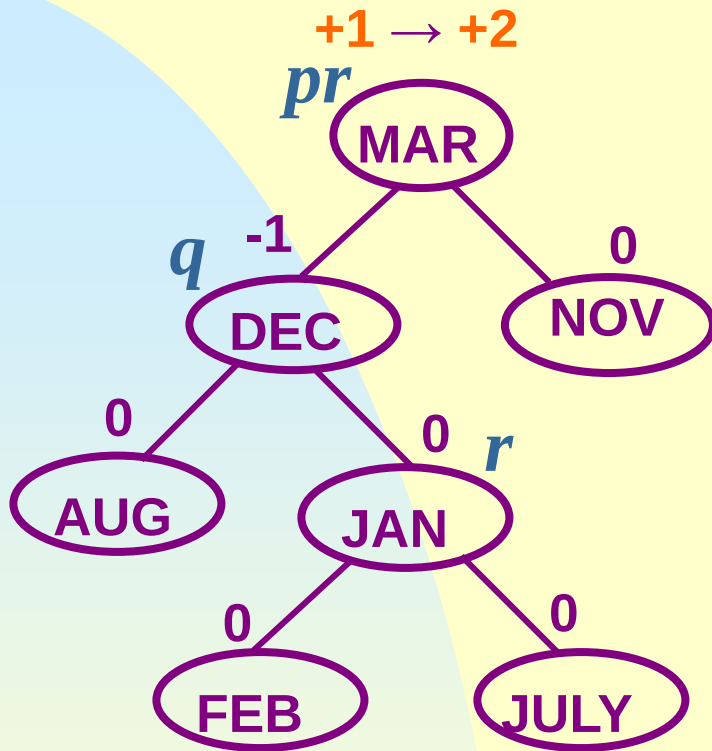
Example: Delete nodes from a AVL Tree



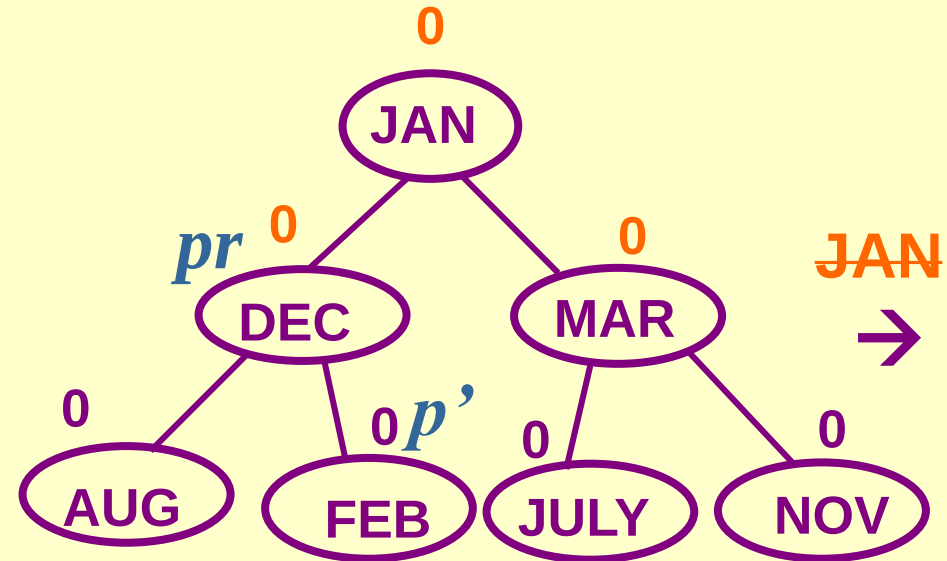
The pointer beside a node represents this pointer points to this node just before the next operation.

(a) After delete APR

Deletion order: APR, MAY, JAN, FEB, DEC, AUG.

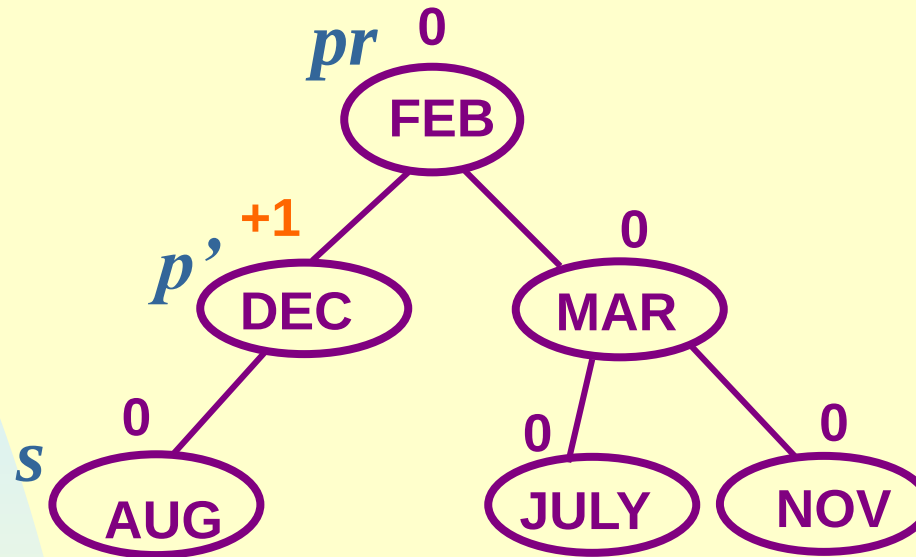


LR
→



(b) After delete MAY

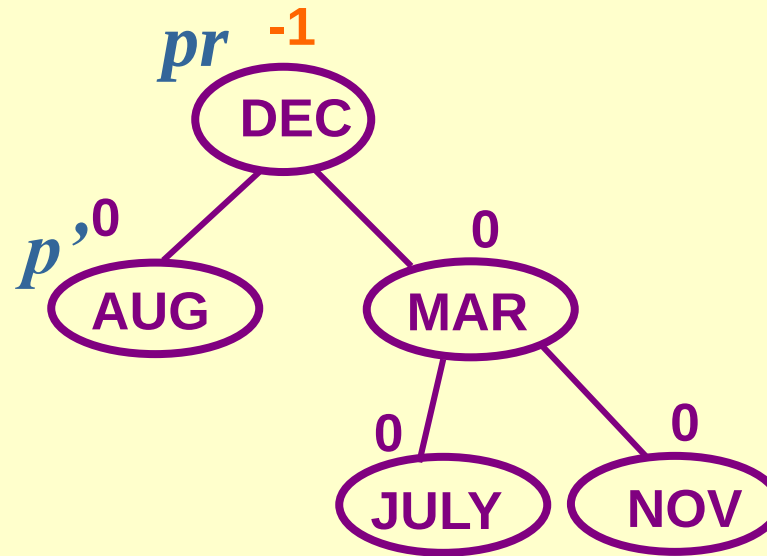
Deletion order: APR, MAY, JAN, FEB, DEC, AUG.



~~FEB~~
→

(c) After delete JAN

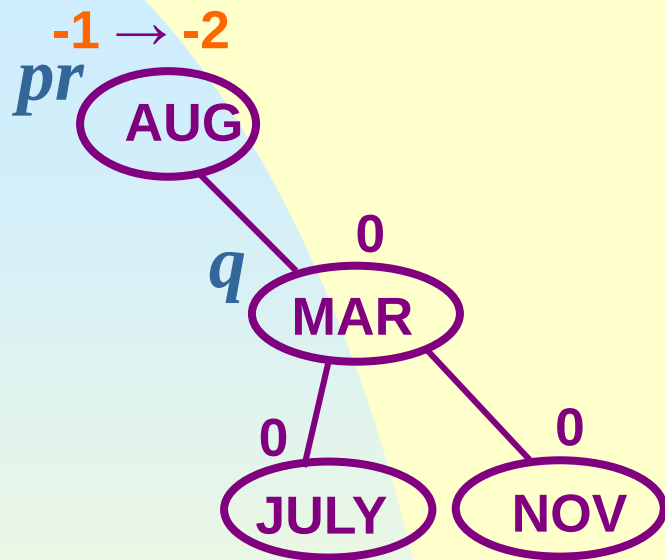
Deletion order: APR, MAY, JAN, FEB, DEC, AUG.



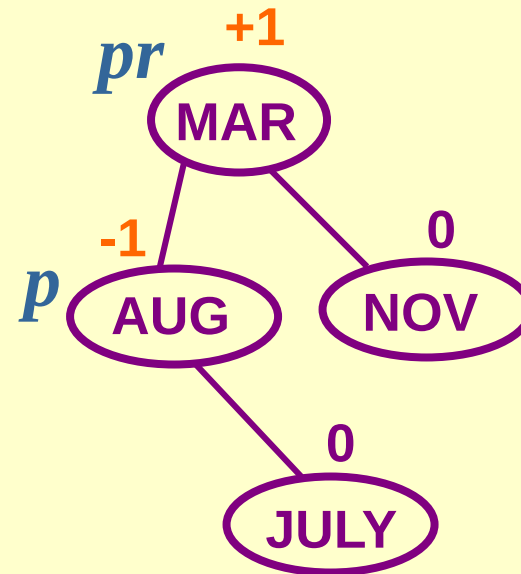
~~DEC~~
→

(d) After delete FEB

Deletion order: APR, MAY, JAN, FEB, DEC, AUG.



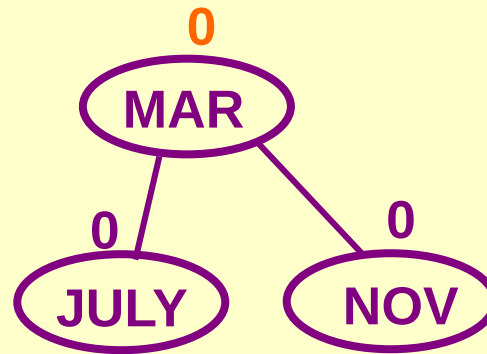
RR
→



~~AUG~~
→

(e) After delete DEC

Deletion order: **APR**, **MAY**, **JAN**, **FEB**, **DEC**, **AUG**.



(f) After delete AUG

Deletion order: APR, MAY, JAN, FEB, DEC, AUG.

■ Analysis of deletion of a node in an AVL tree

- ✓ The time complexity of finding and deletion a node with key k in AVL tree with n nodes is $O(\log n)$.
- ✓ It is also possible to **find and delete** the **i th node** from an AVL tree in $O(\log n)$.

Comparison of the worst-case times of operations on sorted sequential lists, sorted linked lists, and AVL trees

Operation	Sequential list	Linked list	AVL tree
Search for k	$O(\log n)^1$	$O(n)$	$O(\log n)$
Search for jth item	$O(1)$	$O(j)$	$O(\log n)$
Delete element with k	$O(n)$	$O(1)^2$	$O(\log n)$
Delete jth item	$O(n-j)$	$O(j)$	$O(\log n)$
Insert	$O(n)$	$O(1)^3$	$O(\log n)$
Output in order	$O(n)$	$O(n)$	$O(n)$

1. Binary search
2. Doubly linked list and position of k known
3. Position for insertion known

Exercise(s) and Experiment(s)

Exercise(s):

- **P578:9**

Experiment(s):

- **P578:10**